

Course: Foundations of AI with Embeddings

Lecture 7: Visual Analytics of Event Sequences

Multiple Sequence Alignment · Sequen-C visualisation

Liubov B.T., Vladimir Kukushkin

Constructor University, 2026

Multiple Sequence Alignment (MSA)

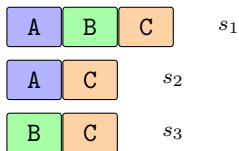
MSA: core idea

Multiple Sequence Alignment was developed in bioinformatics to compare DNA / protein sequences.

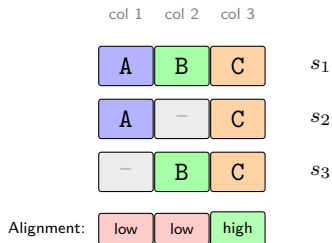
Idea: insert gap characters “-” so that similar amino acids appear in the *same column*.

Why: gaps are associated with evolution changes; the less gaps, the more species are related.

Input sequences (raw):



After MSA (gaps “-” inserted):



Pairwise alignment: the scoring scheme

Goal: given two sequences, find an alignment (via gap insertion) that maximises the total score.

Scoring function $s(a, b)$:

- $s(a, a) = +m$ *match reward*
- $s(a, b) = -x$ *mismatch penalty*
($a \neq b$)
- $s(\cdot, -) = -g$ *gap penalty*

Total alignment score = sum of column-wise scores.

Task: find the alignment with the *highest* total score — i.e., maximise the number of matches and minimise gaps/mismatches.

Example with $m=2$, $x=1$, $g=1$:

s_1	A	B	C	
s_2	A	-	C	
	+2	-1	+2	

score = 3 ✓

s_1	A	B	C	
s_2	A	C	-	
	+2	-1	-1	

score = 0

Needleman-Wunsch algorithm

Needleman-Wunsch (1970) finds the *globally optimal* pairwise alignment in $O(|s_1| \cdot |s_2|)$ time.

Recurrence ($m=2, x=1, g=1$):

$$F[i, j] = \max \begin{cases} F[i-1, j-1] + s(s_1^i, s_2^j) & \text{(diagonal)} \\ F[i-1, j] - g & \text{(up)} \\ F[i, j-1] - g & \text{(left)} \end{cases}$$

Initialisation: $F[0, j] = -jg, \quad F[i, 0] = -ig$

Each cell $F[i, j]$ stores the *best score* for aligning the first i characters of s_1 with the first j characters of s_2 .

Needleman-Wunsch: filling the DP matrix (example)

$s_1=ABC$, $s_2=AC$ ($m=2$, $x=-1$, $g=1$).

Step 1: $F[A, A] = 2$

	ϵ	A	C
ϵ	0	-1	-2
A	-1	2	
B	-2		
C	-3		

diag: $0+2=2 \leftarrow \text{max}$

up: $-1-1=-2$

left: $-1-1=-2$

Step 2: $F[B, A] = 1$

	ϵ	A	C
ϵ	0	-1	-2
A	-1	2	
B	-2	1	
C	-3		

diag: $-1+(-1)=-2$

up: $2-1=1 \leftarrow \text{max}$

left: $-2-1=-3$

Step 3: $F[A, C] = 1$

	ϵ	A	C
ϵ	0	-1	-2
A	-1	2	1
B	-2	1	
C	-3		

diag: $-1+(-1)=-2$

up: $-2-1=-3$

left: $2-1=1 \leftarrow \text{max}$

Step 6: $F[C, C] = 3$

	ϵ	A	C
ϵ	0	-1	-2
A	-1	2	1
B	-2	1	1
C	-3	0	3

diag: $1+2=3 \leftarrow \text{max}$

up: $1-1=0$

left: $0-1=-1$

Needleman-Wunsch: traceback

Traceback: start at $F[|s_1|, |s_2|]$, follow the arrow that produced each cell's value:

- **diagonal** $\nwarrow$: align s_1^i with s_2^j (match or mismatch)
- **up** $\uparrow$: gap in s_2 (s_1^i aligned with "-")
- **left** $\leftarrow$: gap in s_1 (s_2^j aligned with "-")

Result for $s_1=ABC$, $s_2=AC$:

s_1	A	B	C
s_2	A	-	C

score = 3

	ϵ	A	C
ϵ	0	-1	-2
A	-1	2	1
B	-2	1	1
C	-3	0	3

From pairwise to multiple: profiles

Problem: NW works for 2 sequences. How to align $N > 2$ sequences jointly?

Key idea: represent an existing alignment as a **profile** – a frequency matrix.

A **profile** P is an $|\mathcal{A}| \times M$ matrix:

$P_{a,j}$ = fraction of event type a in column j

Profile–sequence score: to align sequence s against profile P , replace the scalar score $s(a, b)$ with a weighted average:

$$s_{\text{profile}}(s^i, P_j) = \sum_{a \in \mathcal{A}} P_{a,j} \cdot s(s^i, a)$$

Then run NW on the profile as if it were a sequence.

This extends naturally to aligning two profiles (profile–profile alignment).

Profile–sequence alignment: worked example

Align $s_3=BC$ with the existing alignment of $s_1=ABC$ and $s_2=A-C$.

Profile P (symbol frequencies per column):

	col 1	col 2	col 3
A	1	0	0
B	0	$\frac{1}{2}$	0
C	0	0	1
–	0	$\frac{1}{2}$	0

NW matrix (rows = s_3 , cols = profile):

	ϵ	P_1	P_2	P_3
ϵ	0	-1	-2	-3
B	-1	-1	-0.5	-1.5
C	-2	-2	-1.5	1.5

Profile–sequence scores $s_{\text{prof}}(s_3^i, P_j)$:

	P_1	P_2	P_3
B	-1	+0.5	-1
C	-1	-1	+2

Result (s_3 gets a gap in column 1):

A	B	C	s_1
A	–	C	s_2
–	B	C	s_3

e.g. $s_{\text{prof}}(B, P_2) = \frac{1}{2} \cdot (+2) + \frac{1}{2} \cdot (-1) = +0.5$

Profile–profile alignment

Similarly, we can align two profiles P and Q using **profile–profile scores**:

$$s_{\text{pp}}(P_j, Q_k) = \sum_{a, b \in \mathcal{A}} P_{a,j} \cdot Q_{b,k} \cdot s(a, b)$$

Profile P (alignment of s_1, s_2, s_3):

	P_1	P_2	P_3
A	$\frac{2}{3}$	0	0
B	0	$\frac{2}{3}$	0
C	0	0	1
–	$\frac{1}{3}$	$\frac{1}{3}$	0

Profile Q ($s_4=AB$):

	Q_1	Q_2
A	1	0
B	0	1
C	0	0
–	0	0

Scores $s_{\text{pp}}(P_j, Q_k)$:	Q_1	Q_2
	P_1	+1 –1
	P_2	–1 +1
	P_3	–1 –1

Merged alignment:

col 1	col 2	col 3	
A	B	C	s_1
A	–	C	s_2
–	B	C	s_3
A	B	–	s_4

e.g. $s_{\text{pp}}(P_2, Q_2) = \frac{2}{3} \cdot s(B, B) + \frac{1}{3} \cdot s(-, B) = 1$

MSA: key properties

Complexity:

- **Two sequences:** $O(|s_1| \cdot |s_2|)$ time
- **Multiple sequences:**
 - No profiles: NP-hard
 - With profiles: $O(M \log N)$

Limitations:

- **Greedy:** early merge errors propagate — no backtracking once a column is aligned
- **Guide tree sensitivity:** different orderings of merges can produce different final alignments
- **Sub-optimal:** the progressive result is not guaranteed to maximise the total alignment score

Sequen-C: The Visualization System

Aggregate tree: hierarchical clustering of sequences

Input: unique sequences $S = \{s_1, \dots, s_N\}$

Goal: build nested clusters.

Algorithm (agglomerative / bottom-up):

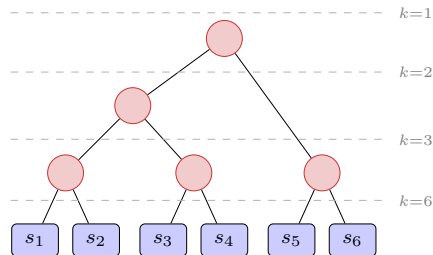
1. Each s_i starts as its own leaf node
2. Compute pairwise distance matrix d :

$$d[i, j] = D_{\text{qgram}}(s_i, s_j)$$

3. Repeat until one root remains:
 - a. Merge the two closest nodes n_a, n_b
 - b. Update d (average linkage)

Distance: cosine distance of **q-gram profiles**

– vectors of sub-sequence counts of length q ;
with $q=1$ order is ignored.



Cutting the tree at height k yields exactly k clusters.
Each internal node stores the merged cluster representation.

Progressive MSA: following the aggregate tree

The **progressive** approach builds alignments bottom-up, following the aggregate tree:

- **Leaf nodes:** alignment λ is the sequence itself
- **Internal node** $n_{a,b}$: align the two child alignments λ_a and λ_b
- At each merge: align two *profiles*
- A profile = $N \times M$ matrix $\rightarrow$ aligning two profiles adds gaps into both

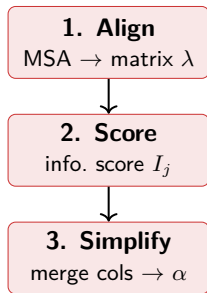
Scoring parameters (scikit-bio):

gap open penalty	0.8
match score	+3
mismatch score	-1

Why progressive? $N - 1$ pairwise profile alignments of sequences of length $\leq M$ gives complexity $O(N \cdot M^2)$ vs. the exact MSA cost $O(M^N)$.

Align → Score → Simplify

Each cluster (tree node) is represented by running three steps on its input sequences:



Step 1 – Align:

Use progressive MSA to align all sequences in the cluster.
Result: $N \times M$ matrix λ (rows = sequences, columns = aligned positions).

Step 2 – Score:

For each column j of λ , compute information score $I_j \in [0, 1]$.
 $I_j = 1$: all events in column j are the same (homogeneous)
 $I_j \approx 0$: column j is mostly gaps or random events

Step 3 – Simplify:

Merge consecutive columns where both $I_j < I_\tau$ and $I_{j+1} < I_\tau$.
Result: simplified matrix α of length $M' \leq M$.
The horizontal level-of-detail is controlled by I_τ .

Score: information score per column

After alignment, measure how **homogeneous** each column j is:

$$I_j = 1 - \frac{E_j}{\log_2(|\mathcal{A}| + 1)}$$

where E_j is the **entropy** of event types in column j :

$$E_j = \sum_{a \in \mathcal{A}_j \cup \{-\}} \begin{cases} -P_a \log_2(P_a/G_j), & a = "-" \\ -P_a \log_2 P_a, & \text{otherwise} \end{cases}$$

- P_a : probability (frequency) of event type a in column j
- G_j : count of gaps in column j (used in gap entropy term)
- $|\mathcal{A}|$: number of event types (alphabet size)
- Column all same event: $I_j = 1$
- Column all gaps or random mix: $I_j \approx 0$

Simplify: merging low-information columns

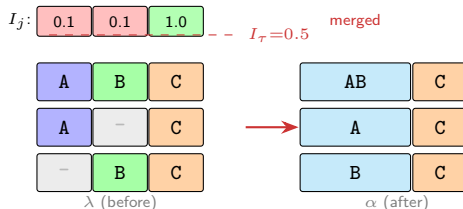
Rule: merge columns j and $j+1$ if *both* $I_j < I_\tau$ and $I_{j+1} < I_\tau$.

How merging works:

- Concatenate characters $\lambda_{i,j}$ and $\lambda_{i,j+1}$ for each row i
- The merged element becomes a *sub-sequence*
- Repeat until no two adjacent columns are both below I_τ

Result: simplified cluster representation α of length $M' \leq M$; increasing $I_\tau \rightarrow$ more merging $\rightarrow$ coarser view.

NB!: This is visible representation, all original λ remain the same.



Full pipeline illustration (from paper)

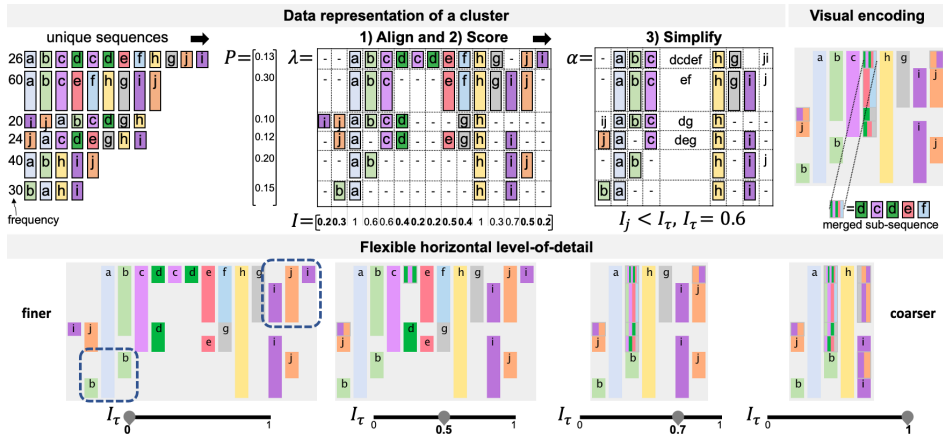


Fig. 3: Cluster data representation (*top left*) and visual encoding (*top right*) for $S = \{s_1, \dots, s_6\}$ and $I_\tau = 0.6$ using the Align-Score-Simplify strategy. The Simplify step produces the simplified alignment matrix α . Using α , the visual encoding is built and event types in the merged sub-sequences are represented using ordered colored bars. Changing I_τ (*bottom*) transforms the horizontal level-of-detail. Note how this shows the most common event orderings (e.g. a-b-c-h) and permutations in the order (e.g. b-a and j-i highlighted with dashed lines).

Aggregate tree illustration (from paper)

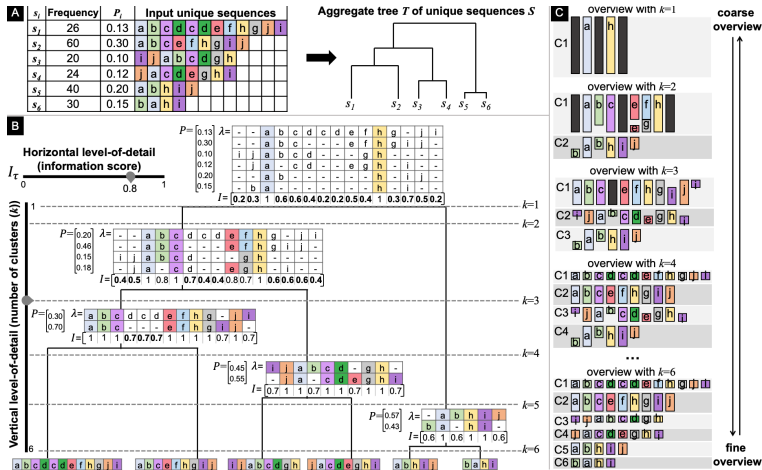


Fig. 2: (A) Building aggregate tree T for input unique sequences $S = \{s_1, \dots, s_6\}$. (B) Each node in T has an alignment matrix λ for its child sequences, a row-wise probabilities vector P , and a column-wise information score vector I . Two or more consecutive columns in λ with $I_j < 0.8$ are not coloured. (C) Multilevel overviews for a range of number of clusters k retrieved from T , where black blocks represent merged columns.

Case study: CUREd — dataset

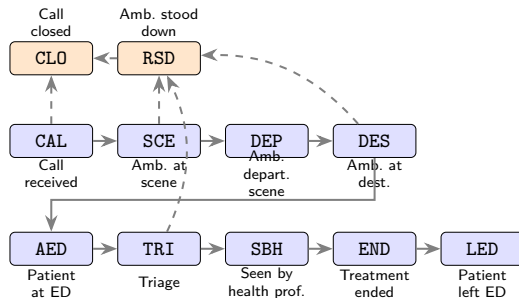
Dataset: CUREd (Centre for Urgent & Emergency Care Research)

Emergency service calls to **999/111** across Yorkshire & the Humber, UK. Three-month subset: **21,805 unique patients**, 25,243 calls, 57 data attributes.

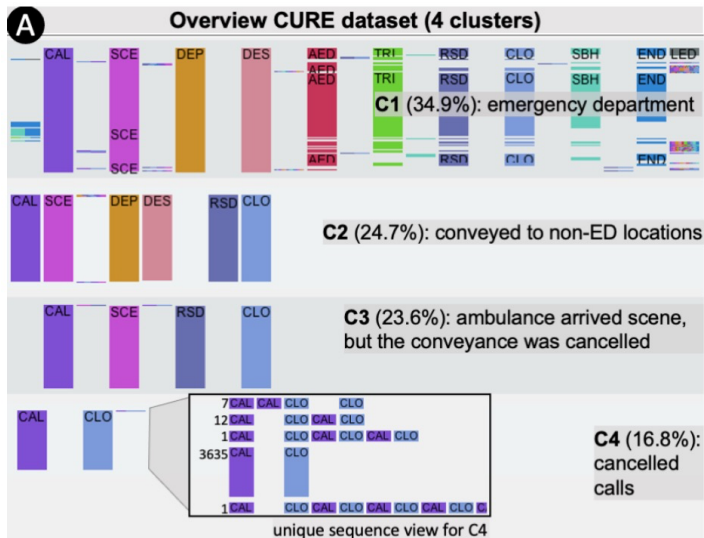
Unique sequences	962
Event types	11
Avg. seq. length	22.7
Max seq. length	177

Each sequence = all events of *all* calls & incidents for one patient.

11 event types — stages of the emergency care pathway:



Case study: CUREd – main clusters overview



Case study: CUREd – cluster C1 overview

